University of Bahrain
College of Information Technology
Department of Computer Science

REAL ESTATE IN BLOCKCHAIN

Prepared by

Adarsh Shinju Chandran 20184747

Ashraf Haris Cheria Valleri 20183717

Shaikh Tahmidur Rahman 20181986

For

ITCS 498

Senior Project

Academic Year 2021-2022-Semester 2

Project Supervisor: Dr. Ali Alsaffar

20/12/2022

Abstract

Due to the complexity and the number of middlemen participants between the buyer and seller of real estate, it introduces frauds, ineffectiveness, unnecessary spendings, and many more. Through the use of blockchain, one can make the real estate process more transparent through the use of distributed ledgers along with smart contracts which are immutable, due to which neither can anyone force a fraudulent transaction, nor try to remove a transaction. We reviewed several research papers that conceptualize the implementation of the Real Estate process on the blockchain. We then study the fundamentals of web3.0 followed by all the various tech that is used in space. Next, we develop algorithms to mimic the real estate transaction and use tokenization of real estate properties to record the buying/selling process as tansaction of NFTs. We conclude that the space is still in its early stage and that more thought needs to be put into accurately creating a model with the proper regulations for real estate transaction and implementing them onto web 3.0.

Acknowledgments

We are immensely grateful to our supervisor Dr. Ali Alsaffar for his assistance and undying support for our project. We are also in depth to Mr Hatem from Black Swan for giving us valuable insight into the Real Estate Market in Bahrain. We would also like to recognize all the Information Technology Doctors who have nurtured us and helped us learn and bear fruit from what we have learnt. Lastly, we would like to thank University of Bahrain and all its staff members and students who have jointly contributed to bringing us to where we are today.

Table of Contents

Chapter 1
Introduction

Real Estate is one of the most lucrative sectors of investment. It is riddled with paperwork, hidden costs, middlemen, regulations, which all contribute to ineffectiveness and frauds. In this project we explore blockchain and its emerging technologies to solve the problems that convention real estate transactions currently face and try to rethink the transaction model for the blockchain. From smart contracts to NFTs, our aim is to explore all the nuances and possibilities that the emerging and highly promising technology can provide, to create a system that is based on trust, safety, privacy, and comfort for all users of the system.

1.1 Problem Statement

Real estate is incredibly complicated. It's time-consuming, complex, risky, and above all else it's expensive. Home inspectors, real estate agents, banks, local governments, and many more, all need to check and sign off on the transaction before it can get done.
The rules of real estate vary widely depending on cities. To navigate such complexities, we can codify local, provincial, and national housing rules and regulations on blockchain smart contracts. The transaction becomes much simpler now since a seller would simply send a location with the desired price and some other ownership information to the contract which would then pull from the ‘database of rules and regulations’ around the sale of real estate in the seller's location. The smart contract could generate contracts, deeds, tax records, anything required to make a sale of the house instantly. A prospective buyer could then meet with the seller agree on pricing in terms and then send their digital signature to the contract verifying their purchase and updating all the documents as required. Through such an application, we aim to lower the barrier of entering the real estate market and reduce the number of middlemen that accompanies a real estate transaction. Additionally, through the use of smart contacts, transactions will be more quick and secure compared to traditional methods.

1.2 Project Objectives

In this project, we will develop an application to facilitate the new landlords and property investors in Bahrain to build their portfolios in the Blockchain environment.
The application should be capable of the following:

1. Deployment of service that accumulates Bahrain real estate information-this will interconnect users with properties that are being sold.
2. Introduce tokenization and create digital identities for each real estate.
3. Sale of properties can be signed, and any obligations arising from the agreement can be tracked by notaries and SLRB.
4. Commercial real estate transactions on the blockchain.

The limitation of our project is that it glosses over the regulations and quality of life features of a public system, because what we intend to make is a proof of concept, that real estate transaction can indeed be done on the blockchain using tokenization of properties. Hence the scope of our project is limited to that which helps us reach our goal.

1.3 Relevance/Significance of the project

The democratization of real estate properties will be possible thanks to blockchain technology. It will no longer be a asset mostly for the elite class, but provide potential investors from all around the world the opportunity to try their skills at real estate investing. The blockchain real estate space will rival the current standard better in terms of efficiency, cost, middlemen hassle, and security. This can be extremely advantageous to the crypto space as well since it will improve the real-world utility of tokens.

1.4 Research Methodology

We have done qualitative research on the various technology associated with blockchain in the field of web3.0. We have gone through prior literature about concepts and ideas about how a real estate system on the blockchain would look like. We also consulted RERA agents who are real estate professionals and surveyed the real estate landscape in Bahrain. Next we implemented smart contracts and tokenization of properties to represent real world non fungible tokens, hence representing a system of buying and selling that is done on the blockchain.

1.5 Scope and limitations

The limitations of our project is that it glosses over the regulations and quality of life features of a public system, because what we intend to make is a proof of concept, that real estate transaction can indeed be done on the blockchain using tokenization of properties. Hence the scope of our project is limited to that which helps us reach our goal.

1.6 Report Outline

The contents of our report will be as follows. Chapter 2 will discuss and do a deep dive on prior work on our domain, thus literature review. Chapter 3 is the methodology followed by Chapter 4 which is the implementation. Here we go through all that is needed to be done to implement and put the system to work. Next, in Chapter 5 we have conclusion and future work.

Chapter 2
Literature Review

Blockchain has become the frontier of decentralisation on the internet, in the past few years. Although the concept of decentralization and distribution of power has been around since the early days of peer to peer sharing and torrents, Satoshi Nakamoto's paper (2008) has been one of the most successful in reviving and driving the collective motivation to build a new decentralised web. The motives for such are endless, from the McDonalds's Monopoly Scam (Bendavid & Harris, 2001) , to the US financial crisis of 2008 that was caused due to negligence and mismanagement by banks (Keats, 2015) . Due to is nature of decentralization, it has been widely used to make decentralized applications (dApps) and bring about a new phase in web technologies, called Web 3.0.

Nakamoto (2008) used the idea of blockchain to implement the decentralized coin called Bitcoin. Instead of a central agency controlling the creation, distribution, and thus the market of the currency, there now exists a network of computers and servers across the globe that verifies transactions and adds successful transactions onto the ledger.

By hashing transactions into a continuous chain of hash-based proof-of-work, the network timestamps transactions, creating a ledger that cannot be modified without repeating the proof-of-work. The longest chain provides evidence for both the order of events seen and the source of the greatest amount of CPU power. The nodes that are not working together to attack the network will produce the longest chain and outperform attackers as long as they control the bulk of the CPU power. There isn't much structure needed for the network itself. Nodes can leave and re-join the network at any time, taking the longest proof-of-work chain as evidence of what transpired while they were away (Nakamoto, 2008) . Modern encryption techniques and its verification procedure can effectively protect the data on blockchain ledgers from illegal access and alteration. Users always have access to a thorough audit trail of activities since the existing "blocks" in the chain can never be replaced.

But what if we could do more than Bitcoin's mission of allowing an unstoppable, decentralized digital payment, and have a platform that facilitated making decentralized software? It could open up the boundaries to more transparency and freedom to control and

govern our own bodies on the internet. This idea has gained a lot of traction after the various privacy breach scandals like that of Facebook-Cambridge Analytica (Grassegger & Krogerus, 2017) or the suppression of news and politicians that speak against the country's reigning government's narrative (Siddiqui & Ghoshal, 2021) . It has raised concerns over a few mega corporations running all the major means of communication on the web and what it means for the population's safety and of its private data, and control over its communication. Thus, Ethereum blockchain was made by its creator Vitalik Buterin.

Buterin wanted to take the blockchain and build a state machine, a global memory that allows people to build programs on top of the blockchain. To do so, the idea of smart Contracts was formed.

1. Smart Contracts

Smart contracts are basically programs that run on the blockchain. These are fundamentally different from the traditional programming language in being permanent. Traditional programs can be modified, the code can be reverse engineered, changed, and the software manipulated. However the idea behind a smart contract is that once the application is deployed, one cannot change the logic of the contract. If one attempts to somehow mess with the contract, the rest of the system rejects the adulterated contract and therefore makes the network self-healing and self-governing.

Smart contracts also allow for multi-sig transaction, requires multiple public keys with their private keys to authorise a transaction (Chainlink, 2022). The contract itself is a set of conditions which when met, a decision gets executed by the contract. This could be something as simple as "when X sends money to Y, transfer money from X wallet to Y's wallet", to a dApp which would feature an entire application. An example of a dApp could be "if user X deposits collateral worth Q amount into the smart contract's wallet, user X can withdraw 50% of Q's worth as loan". This is extremely useful since the central party that holds the collateral, is decentralized and cannot be swayed to act out of agreement. Traditionally, banks and government institutions act as collateral holders but given their centralised nature, it gives them asymmetrical power over the contracts.

One obvious limitation of smart contracts are that they run on the blockchain. Therefore they can only verify conditions that happen within the blockchain. What if the contract wanted to verify something that happens outside the chain, in a real world ? Say person X has received a car from person Y, one would need a trustworthy authority that can verify that X has received

a car from Y in the real world, can then communicate the verification onto the smart contract. This is where hybrid smart contracts come in wherein they use secure middleware called Oracles to combine on-chain events to off-chain infrastructure (Chainlink, 2021).

The security of a blockchain is obtained from a decentralized network of autonomous validators that intentionally restrict their communication with the outside world.

Smart contracts require an additional component of the infrastructure known as an Oracle to safely obtain data from the outside world.

The data that governs Smart contracts must adhere to the strictest specifications since it will ultimately govern the worth of trillions of dollars. Even if we employed a centralized entity to feed information into the smart contract in the real world, it would still result in a singular point of failure, undermining the purpose of a dApp. But what if you could safely access and validate the data that smart contracts require using the same decentralized computing that is used to protect the blockchain? With Chainlink, one service that gives smart contracts a tamper-proof, highly accessible connection to all of the world's top quality data streams, one may go from having one Oracle to a decentralized Oracle. A smart contract using chainlink may receive data on financial markets, cryptocurrency prices, weather, sports scores, IoT sensor readings, and any other real-world authenticated data required to allow more reliable and practical blockchain applications. For high-value smart contract applications on any blockchain network, Chainlink offers the undisputed reality about the actual world, safely tying the existing infrastructure of the real world to the blockchain infrastructure (What Is a Blockchain Oracle?, 2021).

A hybrid smart contract consists of two parts, On-Chain: Blockchain, and Off-Chain: Decentralized Oracle Network. The Blockchain is responsible for maintaining a persistent ledger, approve transactions, and give dispute resolution protocols to help the Off-Chain service. The Decentralized Oracle Network (DON) is responsible for retrieving, validating, and securely delivering the data from the outside source to the smart contract, perform off the blockchain computations, or on the Layer-2 (the crux of which would verified en-mass by the blockchain later on), or do the opposite and relay the result of the smart contract to outside infrastructure. This opens up a new frontier for applications wishing to build on top of the blockchain. External applications such as identify, finance, insurance, etc., require real world data, and significant computation power which is not feasible to put directly onto the blockchain. A hybrid smart contract architecture makes it possible for non-blockchain infrastructure and blockchains to work together in a secure, dependable, scalable, private, adaptable, and/or globally connected way. This opens up frontiers of alliance based on

decentralized systems. The widespread applicability of hybrid smart contracts and Chainlink Decentralized Oracle Networks is a blatant indication that the blockchain ecosystem has only begun to get a glimpse of what is to come, despite the fact that cryptocurrencies are a multi-trillion dollar economy and DeFi is close to a $100B market.

2. Do you need a Real Estate landscape on the blockchain?

Real Estate is an illiquid asset involving many hidden costs during any sort of transaction involving it (Pogue & Miller, 2018). It also involves complex regulations and lots of middle men which make it prone to fraudulence, insecurity, and corruption (Keats, 2015) (Faiaz, 2021). Due to the complexity and lucrative nature of the field, it has many hidden and not so hidden challenges which need to be solved.

2.1. Problem of Title Management

One of the main areas where extortion and errors occur is in title management. In the unlikely event that any defects are found in the property title, the owner must invest much to get them fixed. Additionally, it increases the likelihood that the owner of the property may lose it. Additionally, it forces every property owner to pay money on title insurance to preserve their investment. Block chain offers immutable record management to maintain titles digitally, helping to keep a safe distance from these pressures and avoid danger. It simplifies the procedure, advances simplicity, and saves some money in the process.

2.2. Problem of Liquidity

A portion of the fees for purchasing or renting the property go into the documentation procedure. To avoid any potential legal, financial, and technical concerns in the future, it involves many middlemen from the government office and other agents in the preparation of appraising the property. Since all property paperwork is now done on paper, it is impotent and susceptible to changes, and the record's points of interest can be altered without the genuine owner of the property being informed (CB Insights Research, 2020). Block chain technology and self-executing smart contracts can assist the owners to remain stress-free and save a lot of money by protecting the records from such adjustments and by keeping a record of all the items of interest.

3. Ways to approach the implementation of real estate on blockchain

Because the open ledger system that the block chain uses requires some type of identity to and from which the transactions take place, it is one of the most admirable aspects of conducting business on the block chain because theft may be avoided. For the majority of applications, this identity is formed as a wallet with a special identifying code. This makes the network a platform where parties transferring assets may be verified since when transactions are made, they are validated by users on the network. Everyone has access to the entire copy of the database, similar to how the Bitcoin cryptocurrency's database worked. As contrast to conventional financial institutions or any organization that depends on the database being a centralized one with just a few number of people having access to the data, this makes it hard for hackers to hack and change records. Due to the immutability of blockchain transactions, it is possible to trace back each previous transaction that has been added to a peer-to-peer network node and verified by miners.

3.1. Smart Contracts

Using "smart contracts" is one area where block chain can make a difference. The programs that enable commercial agreements to be carried out in accordance with specific conditions. An example might be a rent or utility agreement where one party pays another party a set amount based on specific factors. Similar to how your bank automatically pays your electricity bill, no matter how high it may be each month, such an instalment payment might also be automated. Smart contracts would reduce the friction involved in conducting the exchange and increase the transparency and fluidity of the real world. Additionally, they would cost less to create and execute compared to outdated financial or real estate contracts, which might lessen the entry barriers for new users.

3.2. Tokenization

Tokenizing fungible resources allows for the faster, easier, and more efficient transfer of assets by purchasing real estate in the real world and converting it into a token. It aids in the digitization of financial units, discretionary sources, and assurances in the land. Programmable sources could be altered to include ownership rights, replacement records, and

rules to ensure valuable asset issuance, scattering, and activities with Ethereum blockchain development.

Additionally, tokenization lowers costs and expedites leasing, buying and selling resources, creating new features, controlling compensation, and managing specific organizational demands. Enhanced liquidity is achieved through increased accessibility across unrivaled sources and connected institutions. These benefits to underwriters and financial specialists demonstrate the validity of block chain in the real estate industry. A tokenized resource resembles a Real Estate Investment Trust (REIT) (Zhou & al, 2020).

3.3. Limitations

Blockchain being a distributed ledger, has its contents publicly available. This drives one of its biggest advantages but also makes it a disadvantage, in certain cases. Seen the blockchain networks are public, anyone can view the transactions, and specific methods can be used to identify the users of a given wallet code. Another challenge of the making a dApp for real estate is that it has many moving parts, many middlemen, regulations which are not only complex but ever changing. On needs to do thorough research before laying down the groundwork for such a project. Additionally, due to the nature of smart contracts, one also needs to think ahead future change in regulations and how it could affect the smart contract. Since the smart contract is distributed and not centrally owned by an entity or a governing body, one needs to pay careful consideration to what kind of details are shared and that security is always given the utmost priority. The technology of tokenization is still relatively new, there are still many things to be discovered and implemented inside it. Work should be done on the security surrounding and the custody of digital tokens. This is because programmers pose a serious threat to electronic tokens. They have enormous destructive potential for both buyers and investors. Tokenized real estate will struggle to establish itself unless institutional-grade governance agreements and trades are understood by the majority.

4. Technologies of web3.0

Due to the ever growing change in landscape of web3.0, and the plethora of solutions that one has to choose from, a section has been dedicated to it, instead of keeping it under ‘Ways to approach the implementation of real estate on blockchain’.

4.1. NFTs

4.1.1. What is an NFT?

A fungible asset in economics is one that has easily interchangeable units. When exchanging money, a $10 note can be exchanged for two $5 notes and remain of same worth. However, if something is non-fungible, this is not possible since it has particular characteristics that prevent it from being substituted for anything else. It may be a home or an exceptional painting like the Last Supper. There is only ever going to be one original artwork, but you can take a picture of it or buy a print. In the digital world, NFTs are "one-of-a-kind" assets that may be purchased and sold just like any other item of property, but they lack a physical form of their own. The digital tokens might be compared to ownership certificates for real or virtual assets. (BBC , 2022)

4.1.2. How do they work?

Paintings and other conventional works of art are valuable because they are unique. However, it is simple and limitless to copy digital files. Using NFTs, a digital certificate of ownership for the artwork may be "tokenized" and traded. A record of who owns what is stored on the blockchain, much like with cryptocurrency. The ledger is maintained by millions of computers all across the world, making it impossible to counterfeit the information. Additionally, NFTs may include smart contracts that, for instance, grant the artist a share of any future token sales. (BBC , 2022)

4.1.3. How to create an NFT?

One would first need to go to the OpenSea NFT marketplace. To produce and mint an NFT, one will also need a cryptocurrency wallet. The act of adding a digital object to the blockchain is known as "minting" an NFT. This proves its unchangeable record of ownership and legitimacy. Cryptocurrency wallets are a key component of web 3.0. In order to communicate with decentralized apps, purchase NFTs, and explore the web3 realm, wallets serve as a private key. To conduct transactions on a certain blockchain, one can utilize a particular wallet. For instance, Phantom is a wallet used to communicate with the Solana blockchain, while MetaMask is a wallet used to communicate with the Ethereum blockchain. Numerous wallets are compatible with OpenSea. An NFT is “minted” and then placed on the

blockchain, where its legitimacy and owner are confirmed. The unchangeable history of an NFT begins with its minting since the blockchain record cannot be changed. (BBC , 2022)

4.1.4. What do they mean for the Real Estate?

Tokenization provides immense benefit when coupled with implementation of real estate on a dApp. It provides unparalleled global liquidity. This may be because it makes cross-border token exchanges simple and extremely safe. Tokens only allow for fragmentary ownership, which is frequently the reason why investing in a real estate project is less expensive. This opens the door for the recruitment of more financial professionals and the consideration of developing economies. The biggest benefit of tokenization is that because all of its forms are stored on a blockchain, there are almost no third parties involved, eliminating any red tape. Blockchain technology provides buyers and investors with such perfect protection that there is no room for any kind of debasement. (Jyotsna & Gampala, 2020)

4.2. IPFS

We are living in a time where the usage of the web has reached its peak, we use the web for learning, watching, communicating with a friend and hosting websites, etc. but the issue with the current web is that it is centralized which means all these information that we get from the web are stored in servers which are controlled by some companies and due to this there arise two problems, first if the company's power supply fails or the place where these servers are stored gets destroyed then no websites will work and the second issue is the censorship as some companies have control of the servers they can remove the access to some website or remove the content from the website. So, the solution for these problems is a decentralized web or IPFS (Interplanetary file system) which is a '*content addressing*' distributed file system and was created by Juan Benet. (IPFS: Interplanetary file storage!, 2018). 'Content addressing' works as follows:

When a file is added to IPFS, it is divided into smaller pieces, cryptographically hashed, and given a special fingerprint known as a content identifier (CID). This CID serves as an immutable record of the file as it was at that time. Other nodes search for the file with the CID, they become another source for the content in the file as they make a copy of the file when they view or download it until their cache is not cleaned. If a node is interested in the content of the file, then that node may pin the content (keep the content forever and will provide it). If the node is not interested and does not pin the content, then it discards the

content to save space. If there is any change made in the file, a new CID is formed, this feature enhances the security as the files on the IPFS become resistant to tampering and censorship. (Potsides, et al., 2022)

When dealing with NFT, traditional HTTP links should be avoided because with an HTTP address like https://cloud-bucket.provider.com/my-nft.jpeg anyone can fetch the contents of *my-nft.jpeg* as long as the server is up. And due to this, there is no more security in the authenticity of the NFT as the content in *my-nft.jpeg* can be changed at any time. In IPFS when an NFT is added a content identifier (CID) that relates to the data in the IPFS network is created straight from the data itself. Since CID can never refer to more than one piece of content, replacing the content is impossible. And as long as there is at least one copy of the data on the IPFS network, anyone can access it using CID even if the original source has vanished. (IPFS, 2022)

4.3. Filecoin

It is a decentralized storage network designed to store the most important information. Filecoin has mechanisms built in to examine the file history and ensure that it has been correctly storing files over time. The Filecoin network is open to everyone and requires no authorization. A spare disk space and an internet connection are all that is needed to run a storage provider. In the decentralized web, Filecoin and IPFS are complementary technologies for storing and exchanging data. While IPFS is optimized for rapid content retrieval and sharing, Filecoin intends to offer longer-term persistence to securely store big quantities of data.

4.4 Graph

The Graph is a decentralized protocol used to index and search data from chains that are EVM-compatible. The Graph coin was produced using the Ethereum Network and is an ERC20 token that can be used to gather information from any ERC20 token. Developers can create numerous APIs known as subgraphs for specific queries using the Graph network. To keep the network's operations running smoothly and protect the blockchain, the Graph network depends on Indexers, Curators, and Delegators. In a decentralized governance paradigm, indexers are given the task of running the nodes and providing best services at the lowest costs possible in the graph market. Curators are given the responsibility of collecting the data and classifying it by relevance and accuracy. And finally, Delegators take up the task of securing the network by entrusting their GRT coins to Indexers. (Eissler, 2022)

4.4. NFT.storage

For off-chain NFT data (such as metadata, photos, and other assets), NFT. Storage is a long-term storage service with a maximum upload size of 31 GB. To make sure the NFT forever truly refers to the desired material, IPFS URLs and CIDs can be utilized in the NFT and metadata. Numerous copies of uploaded data are stored by NFT.storage in two main locations: centralized NFT.storage managed IPFS servers and decentralized Filecoin storage. It is simple to redundantly store data submitted to NFT.storage because IPFS is a standard utilized by many different storage systems. Using the URL (ipfs://<cid>) for off-chain data, for instance, the image field in metadata gives the NFT data protection and cannot be altered due to the usage of CID (from IPFS). Data stored by NFT.storage is accessible from any peer that has the content.

Chapter 3
Methodology

We set onto gain a full understanding of the Real estate market in the Kingdom of Bahrain, the entire process from a person selling his/her property to the buyer buying it and changing the property ownership under the buyer's name. The real estate sector contributes around 6 percent to the Kingdom's GDP and deception in real estate can happen a lot and can affect many people's lives. Blockchain is the way to secure absolutely anything from any fraud so combining blockchain and real estate would eliminate the risk of deception. To combine blockchain and real estate we need not only need to learn the regulations but also the workflow of the transactions. Next we had to learn technical tools and languages to make use of the web3.0 toolkit and implement the process. Below, we will discuss the methods that we used to collect information about the real estate process in Bahrain and how we learnt about blockchain technology.

Interview:

The Interview is the methodology we used to get to know about the real estate market in the Kingdom of Bahrain. We interviewed one of the RERA-certified agents, Mr. Hatem, of one of the top leading real estate agencies in Bahrain, Black Swan which was founded in 2016. Black Swan aspires to offer the best competent and honest real estate consulting services. Their ultimate purpose is not to make sales but to help, support, and locate products affordable for most of their clients.

The entire process of real estate transaction in traditional ways in Bahrain is as follows: -

- First, the seller schedules a meeting with the listing agent to have his property market evaluated, listed, and presented with any disclosures that the buyer will need to make an offer. The seller then signs the listing at the listing agency.
- Next, the agent looks for the buyer, learns about their needs, and if those needs match what the seller is offering, then takes the buyer to see the property.
- If the buyer shows interest in the property and wants to buy it, then the agent and the buyer negotiate the price and reach a final amount at which if the seller does not agree

to sell then the agent must search for a new buyer, but if the seller agrees then agent fixes a meeting between the buyer and the seller.

- Before selling the property the seller must clear all the dues pending but if the property is a flat or an apartment then the seller must also provide a non-objection letter that the committee or the secretary of the building or the management company that manages the building will provide, in which it will be stated that there are no service charges pending for the seller.
- Once all the dues are paid, both the seller and the buyer visit the notary office with the seller's receipts of the payment (and a non-objection letter in case of a flat or an apartment) and can initiate the notarized sales contract process after they have paid the fees to the notary. They can go to the public notary which is SLRB (Survey and Land Registration Bureau) or the private notary and there the notary confirms the payment of the dues by the seller.
- The seller and the buyer must fix an appointment three weeks prior to the meeting with the SLRB whereas the meeting with a private notary can be immediately after the seller pays the pending dues. SLRB charges 20 BHD, whereas the private notary charges more than SLRB, the minimum being between 60 - 70 BHD.
- Then the notary (public or private) provides a notarized contract (the final sales contract) to the seller, making the seller officially ready to sell the property.
- Once the buyer pays the amount to the seller, the notary provides the notarized contract to the buyer. and must register the title deed from the SLRB website.
- After the payment of the amount to the seller, the buyer and the seller must pay 1 percent of the property value to the agent as well.
- Then the buyer can visit the SLRB website and must pay 1.7 percent of the property value to the SLRB (if it is within the first three months of the notarized contract else the buyer must pay 2 percent of the property value to the SLRB) and submit the notarized contract and initiate the title deed changing process.
- The title deed changing process takes about three weeks and then the buyer will get a new title deed under his name and ownership will be transferred.
- Lastly, after getting the title deed in the buyer's name, the buyer can change the EWA (electricity and water authority) meter to his name.

Brad Traversy- We learnt a great deal about ReactJs, and NextJS which are JS framewrks for web development, knowledge of these frameworks helped us go forward with the web implementation of the project.

Free Code Camp- We learned about blockchain technology from freecodecamp's YouTube tutorial videos on blockchain in solidity. The video helped us in our journey and gave us a lot of knowledge on blockchain, solidity, smart contracts, graph, and many more. It is a very beginner-friendly video that made our project journey a lot easier. Whatever we did in the project was all that we learned from the video. (Learn Blockchain, Solidity, and Full Stack Web3 Development with JavaScript – 32-Hour Course, 2022)

Chapter 4
Implementation and Results

4.1 Tools and Programming Languages Used

Choosing useful, efficient and popular tools from an entire plethora of programs and coding languages has been a crucial part of this project. Proper documentation, good community following, and efficiency were the key factors for deciding all the languages used in this project. Some of them are as follows.

- **Visual Studio Code** - is a lightweight and efficient desktop-based source code editor for Windows, macOS, and Linux. It runs multiple languages such as C++, Python, C#, PHP, and Node.js. It helped our project succeed by providing a wealth of useful tools and extensions. There is a highly user-friendly user interface provided by Visual Studio Code. Utilizing extensions from the editor for our project gave us incredibly helpful tools that made it simple to discover code mistakes and use GitHub services.
- **Ethereum Blockchain**- The technology we employed for our project was Ethereum Blockchain. The blockchain was used to achieve the overall goal of simplifying the real estate selling and buying process and reducing the number of intermediaries involved. Ethereum blockchain is the most extensively used blockchain as it provides the creation of smart contracts which are non-editable once written something, so it does secure against any kind of fraud or deception and there is no need for third-party control. In the Ethereum blockchain, Ether cryptocurrency is used to buy real estate property and pay transaction fees. The best blockchain in terms of security is Ethereum because it is the most popular blockchain, which means there are many nodes, and as the number of nodes increases, the harder it becomes to hack thus making it more secure and efficient when compared to other blockchains.
- **Hardhat**- Hardhat is Ethereum software development or Solidity development environment. It is a JavaScript-based framework made up of various parts that may be used to edit, compile, debug, and deploy smart contracts and dApps. The Hardhat's primary component is the Hardhat Runner. It is a versatile and extendable task

manager that aids in organizing and automating the ongoing work necessary for creating smart contracts and dApps. The ideas of tasks and plugins are central to the design of Hardhat Runner. Each time Hardhat is executed from the command line, a task is executed. For instance, the built-in compile job is executed by '*npx hardhat compile.'* Since tasks can call one another, extensive workflows can be defined. Existing tasks can be replaced by users and plugins, allowing users to modify and expand workflows. Two important libraries for the hardhat front-end application are ethers.js and web3.js. We chose ethers.js because it is significantly smaller than web3.js and so loads more quickly and takes up less space Har Hardhat has a plugin to integrate with ethers.js which is called hardhat-ethers. Also, Hardhat contains a built-in Ethereum network node called Hardhat Network intended for development. It enables executing tests, deploying smart contracts, and debugging code all locally on a single workstation. Hardhat also has a plugin, hardhat-etherscan which helps to verify the source code for solidity contracts Etherscan(allows users to easily search and browse transactions and blocks and provides information about each transaction and block, such as the hash and timestamp). Brownie and Truffle are two other popular solidity development environments, but we chose hardhat because of the usage of the console.log () function which is very efficient for a beginner.

- **Solidity**- Solidity is an object-oriented programming language used to run smart contracts on the Ethereum Virtual Machine (EVM) and has syntax similar to C++, Python, and JavaScript. Smart contracts are primarily programmed in Solidity. There are other different smart contract coding languages, solidity is by far the most popular. Solidity files are those with the .sol extension. The four most fundamental data types in Solidity are Boolean, unit, int, address, and bytes. There are many other types as well. The address is a data type that contains address information, such as public wallet addresses. View and pure are two solidity terms that designate a function that does not require gas to operate. We used Solidity in our project to create smart contracts. Solidity is the only language that can be used with the Ethereum blockchain. The main advantage of Solidity is that it provides an Application Binary Interface (ABI) feature that helps to avoid syntax errors.
- **Smart Contracts**- It is the core of our project where we coded the entire process of converting the real estate to NFT (Non-Fungible Token) and making it secure, and transparent and assigned the ownership to the user by their title deed.

- **NFTs**- In our project, we converted the real-estate property into an NFT (Non-Fungible token) and by using metadata we integrated the attributes of the property (like area of the property, location, number of rooms, etc.). Through NFT, ownership of the property will be reassigned when sold.
- **NFT.storage**- It is a free decentralized NFT storage library that ensures decentralization by hosting these files/NFTS in IPFS and Filecoin.
- **Graph-** It is used in our project to search the database or fetch the data from it in a decentralized manner to protect our website and hence implement the decentralization completely**.**
- **NextJS**- Since hardhat is a JavaScript - based framework, the best technology for the front end would be NextJs which is also a JavaScript framework. React is also a JavaScript framework, but we chose nextjs in our project because React supports only client-side rendering on the other hand nextjs supports automatic server rendering and has a feature that automatically gets the code-splitting integrated into the system thus it enhanced the development performance. Some other benefits of using nextjs were enhanced user experience, SEO-friendliness, and optimization.
- **Web3uikit**- It is a lightweight and beautiful UI component for web3 which helps the development of dApps faster. In our project, we used web3uikit's component, a connect button for connecting the wallet addresses of users through which the users could do all transactions.
- **Fleek**- Fleek is a platform for building apps for the decentralized web and hosting websites on IPFS. We used it to host our website. The main advantage of using fleek is that it takes code directly from GitHub so if there were any changes made to the code it will automatically fetch it and display the changes on the website, and the other advantage is due to the usage of IPFS it makes the hosting of websites decentralized.

4.2 Smart Contract Implementation and Testing

Smart contracts are automated programs that can be compiled and run in specific blockchains like Ethereum, Solana, etc. These contracts are almost a replica of real-world contracts, but not trust-based as the latter. that runs in the blockchain. These contracts kind of work the same as classes in the traditional programming languages like C++ and Java. For this project, there are two smart contracts used, PropertyNFT and BlockEstate. These contracts were built

using solidity, the native language for running smart contracts in any EVM (Ethereum Virtual Machine) based chains. The following are the code of the smart contracts used.

PropertyNFT Contract

```
//SPDX-License-Identifier: MIT
pragma solidity ^0.8.9;

// Import Statements
import "@openzeppelin/contracts/access/Ownable.sol";
import
"@openzeppelin/contracts/token/ERC721/extensions/ERC721URIStorage.so
l";
import "@openzeppelin/contracts/utils/Counters.sol";

// Custom Errors
error PropertyNFT__NotEnoughETHToMint();
error PropertyNFT__TransferFailed();
error PropertyNFT__UserIsNotSlrb();

/** @title A ERC721 Token that represents a property asset
    @dev A ERC721 based Contract that can mint and burn custom
metadata NFTs
*/
contract PropertyNFT is ERC721URIStorage, Ownable {
    // Counters to keep track of TokenID
    using Counters for Counters.Counter;
    Counters.Counter private _tokenIds;

    // State Variables //
    uint256 private immutable i_mintFee;
    address private immutable i_slrb;

    // Events //
    event PropertyMinted(uint256 tokenId, address owner);
    event PropertyExpired(uint256 tokenId);

    // Modifiers //
    modifier onlySlrb() {
        if (msg.sender != i_slrb) {
            revert PropertyNFT__UserIsNotSlrb();
        }
        _;
    }

    // constructor //
    constructor(
        uint256 _mintFee,
```

```
        address _slrb
    ) ERC721("BlockEstate NFTs", "BEN") {
        i_mintFee = _mintFee;
        i_slrb = _slrb;
    }

    // Main Functions //
    /**
     * @dev Method for minting Property NFT
     * @notice Mintfee is colled for minting Property NFT
     * @param tokenURI TokenURI of Property NFT
     * @param propertyOwner The Address of the owner who owns the
Property asset
     */
    function mintPropertyNFT(
        string memory tokenURI,
        address propertyOwner
    ) public payable onlySlrb returns (uint256) {
        if (msg.value < i_mintFee) {
            revert PropertyNFT__NotEnoughETHToMint();
        }
        _tokenIds.increment();
        uint256 latestPropertyID = _tokenIds.current();
        _mint(propertyOwner, latestPropertyID);
        _setTokenURI(latestPropertyID, tokenURI);
        emit PropertyMinted(latestPropertyID, propertyOwner);
        return latestPropertyID;
    }

    /**
     * @dev Method for burning the old Property NFT during the
transfer of ownership
     * @param _tokenId The old token ID of the Property NFT thats
going to be burnt
     */
    function expirePropertyNFT(uint256 _tokenId) public onlySlrb {
        _burn(_tokenId);
        emit PropertyExpired(_tokenId);
    }

    /**
     * @dev Method to withdraw funds that was collected during the
minting process.
     * @notice This can only be invoked by the owner
     */
    function withdraw() public onlyOwner {
        uint256 amount = address(this).balance;
```

```
        (bool success, ) = payable(msg.sender).call{value:
amount}("");
        if (!success) {
            revert PropertyNFT__TransferFailed();
        }
    }

    /**
     * @dev Method to get the fees charged for minting Property NFT.
     */
    function getMintFee() public view returns (uint256) {
        return i_mintFee;
    }

    /**
     * @dev Method to get the current TokenID of the Property NFT.
     */
    function getTokenCounter() public view returns (uint256) {
        return _tokenIds.current();
    }

    /**
     * @dev Method to reset the TokenID to 0.
     * @notice Method can only be called by the owner
     * @notice Will be only used for testing and in default scripts
     */
    function resetCounter() public onlyOwner {
        _tokenIds.reset();
    }
}
```

The PropertyNFT contract implements an ERC-721 based token, commonly known as NFTs (Non-fungible tokens). These are special smart contracts that require some specific functions that should be implemented, following the EIC-721 standard. In this case, NFTs are used as an proof of ownership, where the property is linked to the owner's wallet address. These NFTs also contain custom metadata for each of the token ids, even though all of them have the same NFT address. Some of the custom metadata include the title deed document, property images, property type, property area and much more. The main image of the NFT is the company logo. All the title deeds, property images, and the main image have been hosted on IPFS and Filecoin, to ensure decentralization. This was done using NFT.Storage, which is a free decentralized storage and bandwidths for NFTs.

For this PropertyNFT Contract, openzepplin contracts were used in getting a lot of pre-defined functions for ERC-721 tokens. Moreover, a counter variable was also used from the

openzepplin Counters contract, to keep track of the token Ids of the Property NFT. The mintProperty function is triggered, whenever SLBR creates on new PropertyNFT for the owner, or when there's a transfer of ownership of the property (when someone buys the property from BlockEstate in this case). And whenever there's a transfer of ownership, the old PropertyNFT which belongs to the previous buyer, will be burnt.

Block Estate Contract

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.9;

// Import Statements
import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "./PropertyNFT.sol";

// Custom Errors
error BlockEstate__PropertyAlreadyForSale(uint256 tokenId);
error BlockEstate__PropertyNotOnSale(uint256 tokenId);
error BlockEstate__UserIsNotAdmin();
error BlockEstate__UserIsNotNotary();
error BlockEstate__UserIsNotSlrb();
error BlockEstate__NotOwner();
error BlockEstate__ETHMustBeMoreThanZero();
error BlockEstate__NoInterestOnPropertyYet(uint256 tokenId);
error BlockEstate__PropertyAlreadyVerified(uint256 tokenId);
error BlockEstate__UserIsNotTheInterestedBuyer();
error BlockEstate__NotEnoughETH(uint256 tokenId, uint256 price);
error BlockEstate__NoETHToWithdraw();
error BlockEstate__ETHTransferFailed();

contract BlockEstate is ReentrancyGuard {
    enum Status {
        LISTED,
        INTERESTED,
        VERIFIED,
        TRANSFERRING,
        CANCELLED
    }

    // Type Declarations //
    struct Listing {
        address seller;
        address buyer;
        uint256 price;
        Status status;
    }
```

```
    // Events //
    event PropertyListed(
        address indexed seller,
        uint256 indexed tokenId,
        uint256 price
    );

    event PropertyCanceled(address indexed seller, uint256 indexed
tokenId);

    event PropertyInterested(address indexed buyer, uint256 indexed
tokenId);

    event PropertyVerified(uint256 tokenId);

    event PropertyTransaction(
        address indexed buyer,
        uint256 indexed tokenId,
        uint256 price
    );

    event PropertyOwnershipTransfer(uint256 oldTokenId, uint256
newTokenId);

    // State Variables //
    PropertyNFT private immutable i_propertyNFT;
    address private immutable i_admin;
    address private immutable i_notary;
    address private immutable i_slrb;
    mapping(uint256 => Listing) private s_listings;
    mapping(address => uint256) private s_proceeds;

    // Modifiers //
    modifier notOnSale(uint256 tokenId) {
        Listing memory listing = s_listings[tokenId];
        if (listing.price > 0) {
            revert BlockEstate__PropertyAlreadyForSale(tokenId);
        }
        _;
    }

    modifier onSale(uint256 tokenId) {
        Listing memory listing = s_listings[tokenId];
        if (listing.status > Status.LISTED) {
            revert BlockEstate__PropertyNotOnSale(tokenId);
        }
        _;
    }
```

```
    modifier onlyAdmin() {
        if (msg.sender != i_admin) {
            revert BlockEstate__UserIsNotAdmin();
        }
        _;
    }

    modifier onlyNotary() {
        if (msg.sender != i_notary) {
            revert BlockEstate__UserIsNotNotary();
        }
        _;
    }

    modifier onlySlrb() {
        if (msg.sender != i_slrb) {
            revert BlockEstate__UserIsNotSlrb();
        }
        _;
    }

    modifier isOwner(uint256 tokenId, address buyer) {
        address owner = i_propertyNFT.ownerOf(tokenId);
        if (buyer != owner) {
            revert BlockEstate__NotOwner();
        }
        _;
    }

    // Constructor //
    constructor( address _propertyNFTAddress, address _notary,
address _slrb) {
        i_propertyNFT = PropertyNFT(_propertyNFTAddress);
        i_admin = msg.sender;
        i_notary = _notary;
        i_slrb = _slrb;
    }

    // Main Functions //
    /**
     * @dev Method for Listing property on sale
     * @param tokenId Token ID of Property NFT
     * @param price sale price (in ETH) for each property
     */
    function sellProperty(
        uint256 tokenId,
        uint256 price
```

```
    ) external notOnSale(tokenId) isOwner(tokenId, msg.sender) {
        if (price <= 0) {
            revert BlockEstate__ETHMustBeMoreThanZero();
        }
        s_listings[tokenId] = Listing(
            msg.sender,
            address(0),
            price,
            Status.LISTED
        );
        emit PropertyListed(msg.sender, tokenId, price);
    }

    /**
     * @dev Method for cancelling the property sale
     * @notice Property sale cannot be cancelled if the notary
completed the verification process
     * @param tokenId Token ID of the Property NFT
     */
    function cancelPropertySale(
        uint256 tokenId
    ) external isOwner(tokenId, msg.sender) onSale(tokenId) {
        delete (s_listings[tokenId]);
        emit PropertyCanceled(msg.sender, tokenId);
    }

    /**
     * @dev Method to show that the user is interested to buy the
property
     * @param tokenId Token ID of Property NFT
     */
    function propertyInterested(uint256 tokenId) external
onSale(tokenId) {
        s_listings[tokenId].status = Status.INTERESTED;
        s_listings[tokenId].buyer = msg.sender;
        emit PropertyInterested(msg.sender, tokenId);
    }

    /**
     * @dev Method for verifying the details of the property by the
notary
     * @param tokenId Token ID of Property NFT
     */
    function propertyVerified(uint256 tokenId) external onlyNotary {
        Listing memory property = s_listings[tokenId];
        if (property.status == Status.LISTED) {
            revert BlockEstate__NoInterestOnPropertyYet(tokenId);
        } else if (property.status > Status.INTERESTED) {
```

```
            revert BlockEstate__PropertyAlreadyVerified(tokenId);
        }
        s_listings[tokenId].status = Status.VERIFIED;
        emit PropertyVerified(tokenId);
    }

    /**
     * @dev Method for the buyer to pay the amount of the property
to the old owner
     * @param tokenId Token ID of Property NFT
     */
    function payPropertySaleAmount(
        uint256 tokenId
    ) external payable nonReentrant {
        Listing memory listedProperty = s_listings[tokenId];
        if (msg.sender != listedProperty.buyer) {
            revert BlockEstate__UserIsNotTheInterestedBuyer();
        }
        if (msg.value < listedProperty.price) {
            revert                  BlockEstate__NotEnoughETH(tokenId,
listedProperty.price);
        }
        s_proceeds[listedProperty.seller] += msg.value;
        s_listings[tokenId].status = Status.TRANSFERRING;
        emit         PropertyTransaction(msg.sender,         tokenId,
listedProperty.price);
    }

    /**
     * @dev Method for transferring the ownership of the Property
     * @param oldTokenId Old Token ID of Property NFT
     * @param tokenURI for minting a new Property NFT
     */
    function transferOwnership(
        uint256 oldTokenId,
        string memory tokenURI
    ) external onlySlrb returns (uint256) {
        Listing memory oldProperty = s_listings[oldTokenId];
        uint256 newTokenId = i_propertyNFT.mintPropertyNFT(
            tokenURI,
            oldProperty.buyer
        );
        i_propertyNFT.expirePropertyNFT(oldTokenId);
        delete (s_listings[oldTokenId]);
        emit PropertyOwnershipTransfer(oldTokenId, newTokenId);
        return newTokenId;
    }
```

```
    /**
     * @dev Method for withdrawing ETH from the sales
     */
    function withdrawProceeds() external {
        uint256 proceeds = s_proceeds[msg.sender];
        if (proceeds <= 0) {
            revert BlockEstate__NoETHToWithdraw();
        }
        s_proceeds[msg.sender] = 0;
        (bool success, ) = payable(msg.sender).call{value:
proceeds}("");
        if (!success) {
            revert BlockEstate__ETHTransferFailed();
        }
    }

    /**
     * @dev Method to check if its a regular user that wants to buy
/ sell / view properties
     */
    function isRegularUser() public view returns (bool) {
        address user = msg.sender;
        return (user != i_admin && user != i_notary && user !=
i_slrb);
    }

    /**
     * @dev Method to check if the user is the admin
     */
    function isAdmin() public view returns (bool) {
        return (msg.sender == i_admin);
    }

    /**
     * @dev Method to check if the user is the admin
     */
    function isNotary() public view returns (bool) {
        return (msg.sender == i_notary);
    }

    /**
     * @dev Method to check if the user is the admin
     */
    function isSlrb() public view returns (bool) {
        return (msg.sender == i_slrb);
    }

    // Getter Functions //
```

```
    function getListedProperty(
        uint256 tokenId
    ) external view returns (Listing memory) {
        return s_listings[tokenId];
    }

    function getProceeds(address seller) external view returns
(uint256) {
        return s_proceeds[seller];
    }
}
```

The BlockEstate contract is the main core of this project, similar to some processing unit in a hardware system. It has many functions and events from which, both the frontend and the graph will be able to retrieve the required information or use the function itself. Almost the entire real estate process in Bahrain have been implemented in this contract in a simplified manner. There are four user roles in the contract: Admin, Notary, SLRB and Regular User. The admin is the person who deploys the contract, and the only person that can withdraw all the fees that was collected for various procedures in the real estate process. Other than the verification process and minting / burning Property NFTs, every other tasks can be done by the Admin. Whereas Notary will be able to verify if the property has every tax or loan cleared, if there's any amount of rent needed to be paid and more. Once the verification process is completed, the buyer will be able to pay to the seller. In the case of SLRB, the organization will be able to issue and remove Property NFTs, when there's a transfer of ownership. SLRB will also be able to mint new Property NFTs, and will be distributed to the owner's private wallet. And lastly, in the case of Regular user, he/she can view, buy or sell properties that they are interested in. The buyer if interested in buying a property, can ping the seller, which will then start the entire real estate process, starting with the verification.

An Open zeppelin contract Reentrancy Guard was used in order to prevent Reentrancy attacks from happening in the contract. Multiple custom errors were also added to check various validations in the program. Events such as PropertyListed, PropertyCancelled, PropertyInterested, PropertyVerified, PropertyTransaction, and PropertyOwnershipTransfer were included so that, the corresponding entities in the Graph file can be updated. Every function was carefully created with needed modifiers in order to avoid authorization for unauthorized users.

```
PS C:\Users\lachi\Documents\PROJECT\RealEstateBlockchainApp> yarn hardhat deploy --network localhost
yarn run v1.22.19
$ C:\Users\lachi\Documents\PROJECT\RealEstateBlockchainApp\node_modules\.bin\hardhat deploy --network localhost
Nothing to compile
No need to generate any newer typings.
**********

reusing "PropertyNFT" at 0x8A791620dd6260079BF849Dc5567aDC3F2FdC318
reusing "BlockEstate" at 0x610178dA211FEF7D417bC0e6FeD39F05609AD788
Providing access to BlockEstate to mint and burn Property NFTs
Access has already been provided!

**********
Updating contract address to Frontend...
Updating ABIs to Frontend...
Updated to the Frontend

**********
Done in 2.66s.
```

Figure 4.2.1 Deploying both contracts to the local network

```
$ C:\Users\lachi\Documents\PROJECT\RealEstateBlockchainApp\node_modules\.bin\hardhat deploy --network goerli
Nothing to compile
No need to generate any newer typings.
**********

reusing "PropertyNFT" at 0x4e9Ff08d1d2832F4F614Afbc75CdF5C47b674C35
deploying "BlockEstate" (tx: 0x4b7fa767414e646f8e328749144d8168976c45faf1b735bbbc8b0945f7718fd8)...: deployed at 0xFa00C473cecf4
0AD58732D00939D1A7011e7db48 with 2074894 gas
Verifying contract...
Generating typings for: 18 artifacts in dir: typechain-types for target: ethers-v5
Successfully generated 48 typings!
Compiled 18 Solidity files successfully
Successfully submitted source code for contract
contracts/BlockEstate.sol:BlockEstate at 0xFa00C473cecf40AD58732D00939D1A7011e7db48
for verification on the block explorer. Waiting for verification result...

Successfully verified contract BlockEstate on Etherscan.
https://goerli.etherscan.io/address/0xFa00C473cecf40AD58732D00939D1A7011e7db48#code

**********
Updating contract address to Frontend...
Updating ABIs to Frontend...
Updated to the Frontend

**********
Done in 119.61s.
PS C:\Users\lachi\Documents\PROJECT\RealEstateBlockchainApp>
```

Figure 4.2.2 Deploying both contracts to Ethereum Testnet Goerli along with verification

For deploying the scripts, a dependency called ‘hardhat-deploy’ which was created by the hardhat community was used. This really helped to make the code more structured, and provided with some out-of-the-box solutions like named accounts, which was beneficial for the user-role based function access. The contracts, that were deployed to Ethereum test-net (Goerli in this case) was also verified programmatically, using the etherscan api key. Moreover, both the contract address and the abi generated after the deployment is written to the file in the frontend (/client directory), so that all the public contract functions can be triggered in the frontend.

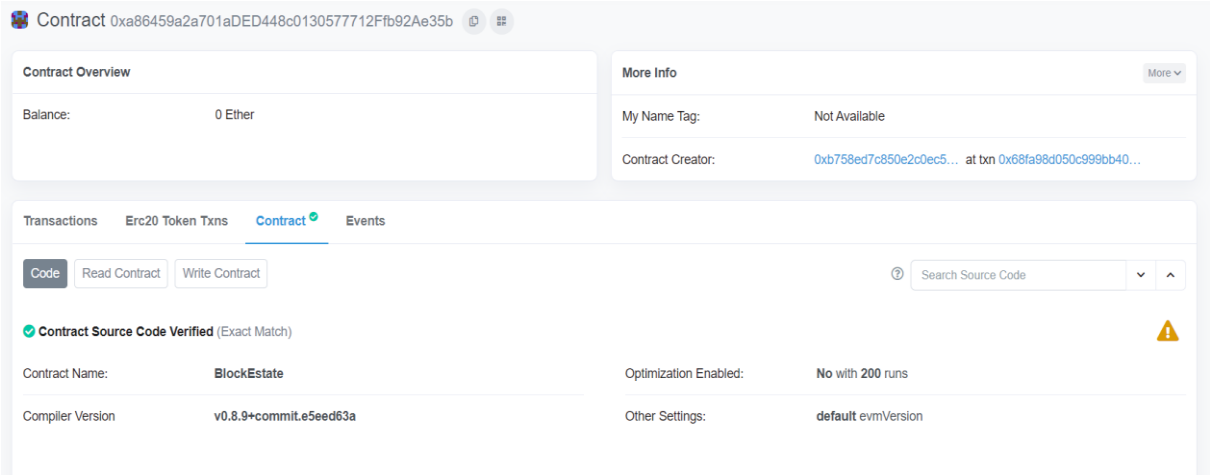

Figure 4.2.3 BlockEstate contract creation transaction on Etherscan and proof of verification

Both the contracts have followed all the major conventions in writing the solidity language, mainly referred from the official Solidity documentation. Some other conventions that was followed include prefixing ‘s_’ on variables that use storage, prefixing ‘i_’ for immutable variables etc. Moreover, some best practices for gas optimizations was also included in both of the contracts. Some of them include using constant and immutable to variables that are not going to change its value after the initialization. Another one is the usage of custom errors than the more traditional usage of require function, which is much more gas expensive. With the help of coinmarketcap API and hardhat-gas-reporter dependency, a text file was created that contains all the information regarding the gas fees in BHD (Bahraini Dinar) and amount of gas used.

gas-report.txt

Solc version: 0.8.9	Optimizer enabled: false	Runs: 200	Block limit: 30000000 gas
Methods	19 gwei/gas		458.73 bhd/eth

Contract	Method	Min	Max	Avg	# calls	bhd (avg)
PropertyNFT	setApproveForAll	27179	49319	46189	18	0.40
PropertyNFT	mintProperty	-	-	92541	15	0.81
RealEstate	buyItem	-	-	89976	2	0.78
RealEstate	cancelProperty	-	-	33013	1	0.29
RealEstate	sellProperty	-	-	80545	10	0.70
RealEstate	updateListing	-	-	41441	1	0.36
RealEstate	withdrawProceeds	-	-	28747	2	0.25

Deployments				% of limit	
PropertyNft	-	-	2174485	7.2 %	18.95
BlockEstate	-	-	1387602	4.6 %	12.09

Figure 4.2.4 Gas Report Generated with Gas Fees and Average Amount in BHD

Additionally, some default scripts were also created to mint, list and cancel property for easier testing of the application. These propertyNFTs, that were minted was stored in NFT.storage (a IPFS and Filecoin based Decentralized storage service) and the transaction can be verified on etherscan and openSea (one of the most popular NFT marketplace).

Figure 4.2.5 Minting 6 Property NFTs and Storing it in NFT.storage

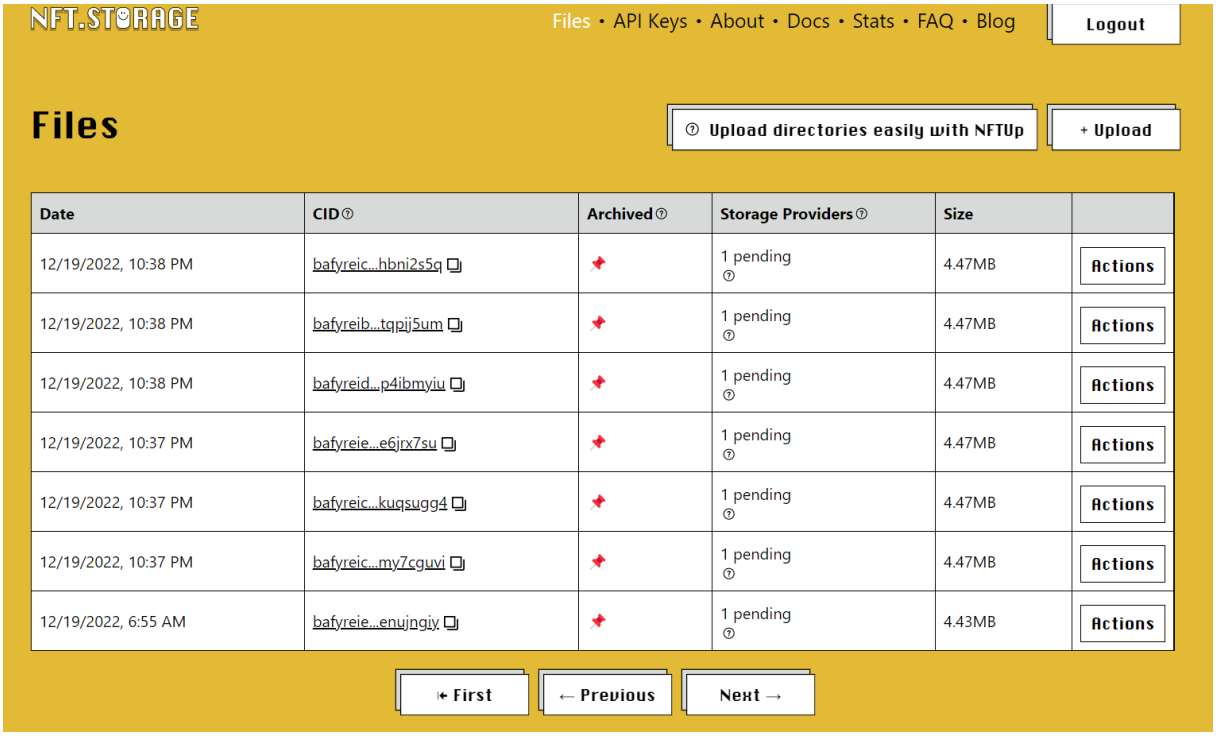

Figure 4.2.6 Property NFTs stored programmatically in NFT.Storage and viewed in NFT.Storage dashboard

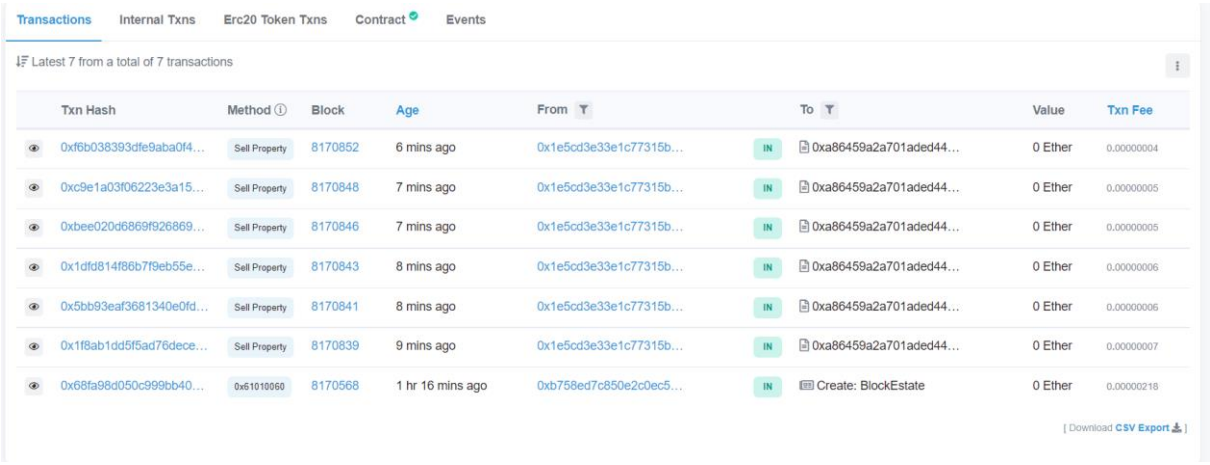

Figure 4.2.7 A list of transactions to list/sell the property on BlockEstate from etherscan

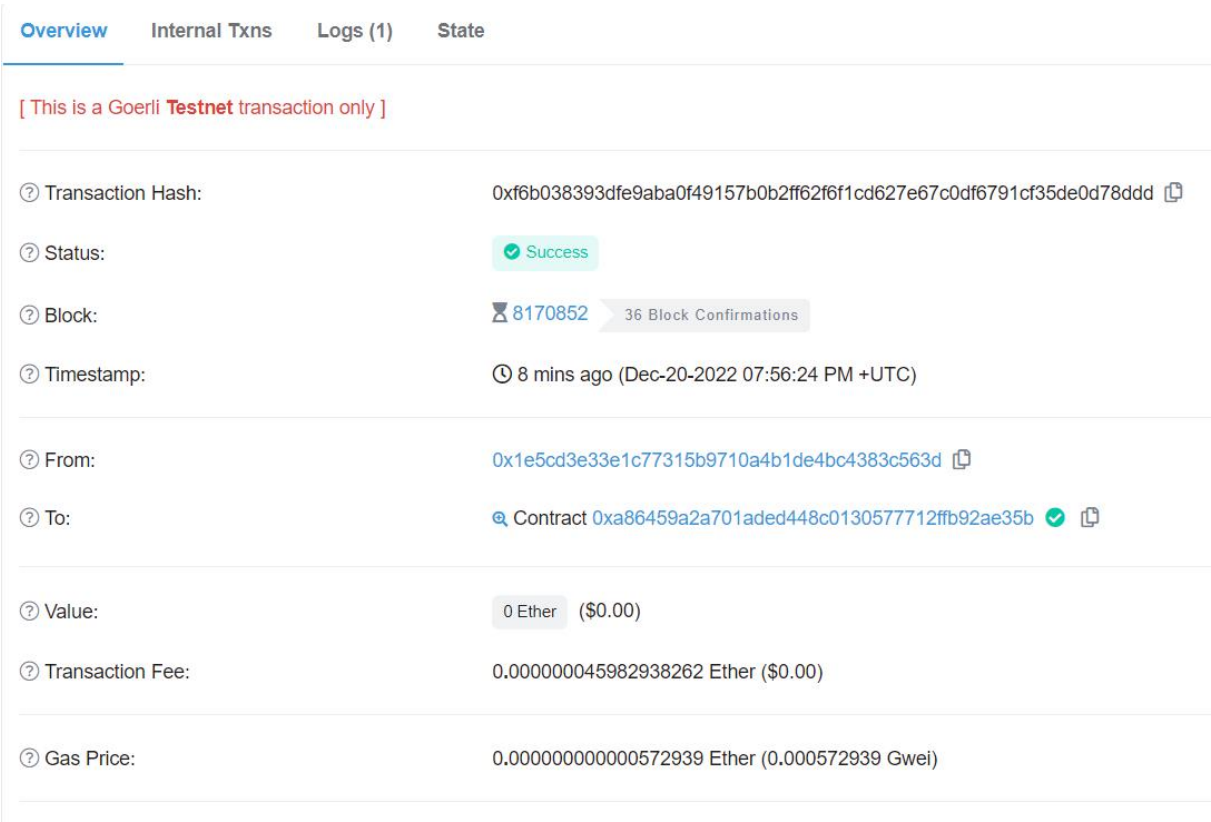

Figure 4.2.8 A single transaction when the Property is listed in BlockEstate from Etherscan

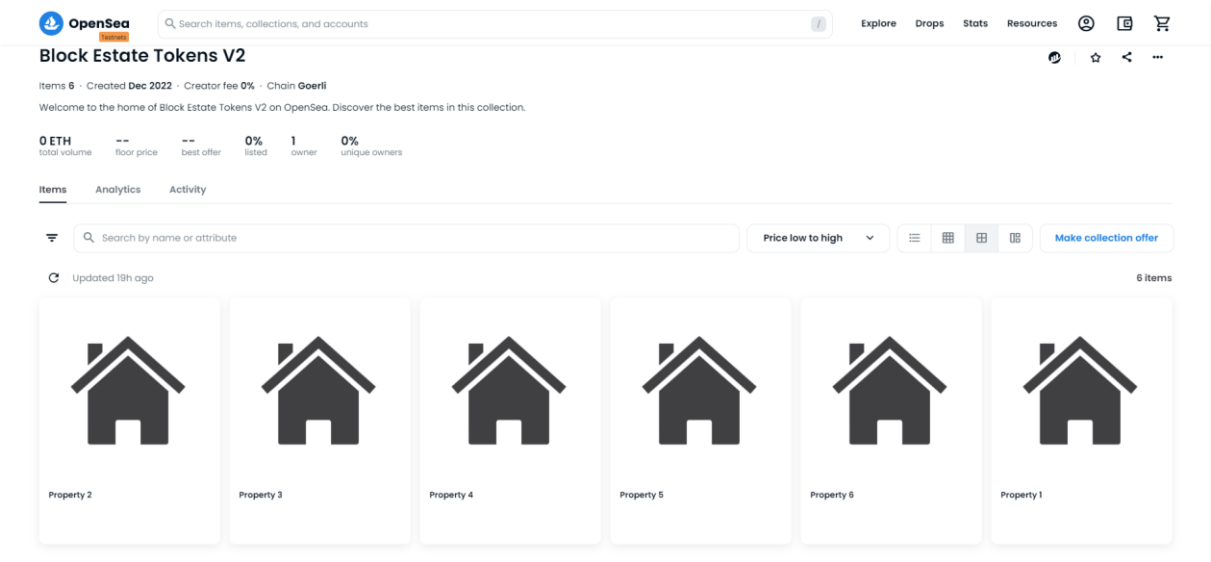

Figure 4.2.9 Few Minted Collection of Property NFTs on OpenSea

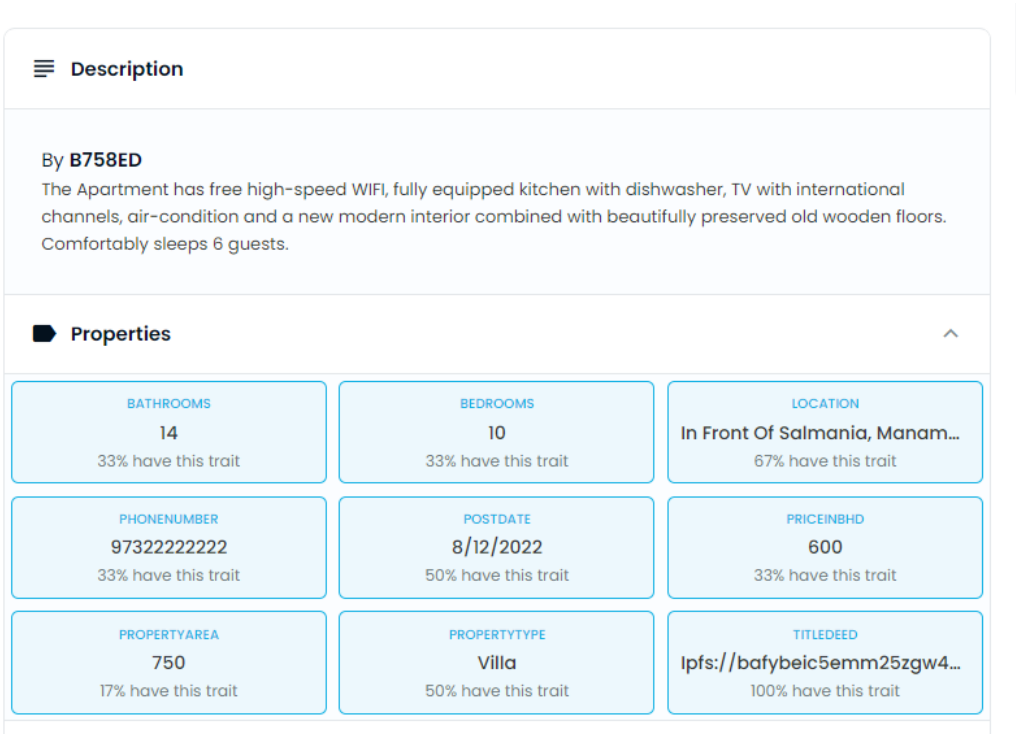

Figure 4.2.10 Metadata of a single Property NFT on OpenSea

Testing is one of the most important part of a blockchain based software system, as it is strictly necessary to avoid unnecessary expenses unlike traditional web development. Testing for this smart contract was done using the inbuilt Hardhat testing workflow, which mainly comprises of Waffle and chai. Extensive testing was done to make sure if that contract was working without any malfunctioning.

```
PS C:\Users\lachi\Documents\PROJECT\Test\RealEstateBlockchainApp> yarn hardhat test
yarn run v1.22.19
$ C:\Users\lachi\Documents\PROJECT\Test\RealEstateBlockchainApp\node_modules\.bin\hardhat test

  Nft Marketplace Unit Tests
    sellProperty
      √ emits an event after listing a property for sale
      √ exclusively properties that are not for sale
      √ exclusively allows owners to list property
      √ needs approval to BlockEstate to list a property
      √ Updates PropertyListed with seller and price
    cancelProperty
      √ reverts if there is no property for sale
      √ reverts if anyone but the owner tries to call
      √ emits event and removes the PropertyNFT from Listing
    buyProperty
      √ reverts if the Property isn't for sale
      √ reverts if the price isnt met
      √ transfers the nft to the buyer and updates internal proceeds record
    PropertyVerified
      √ must be a Notary and for sale
      √ verifies the property
    withdrawProceeds
      √ doesn't allow 0 proceed withdrawals
      √ withdraws proceeds


  15 passing (2s)

Done in 7.26s.
PS C:\Users\lachi\Documents\PROJECT\Test\RealEstateBlockchainApp>
```

Figure 4.2.11 Unit Testing using Hardhat testing setup (specifically chai)

Since querying data from the blockchain network is extremely complex and too expensive, using a database or some indexing protocol will be a necessity as that data is required to be shown on the frontend. But the major issue with any regular database like mysql, firebase, mongodb or even moralis (similar to firebase but in the web3 space) is that all of them are centralized. If the central server goes down, data is compromised. Therefore, for this project, the graph was chosen. The graph is an indexing protocol in networks like Ethereum or IPFS, which helps in querying data, ensuring a fully decentralized experience. But there are unfortunately some tradeoffs, which includes scalability, speed and querying complex queries when compared to traditional databases.

A new folder was set up as it was required by the graph. It used graphql API to query the data, and entities were made using the graph query. The graph studio and its playground were used extensively for the documentation and testing of various queries. A dependency called @apolo/client was used to access and use the graph queries in the frontend to fetch data to the website.

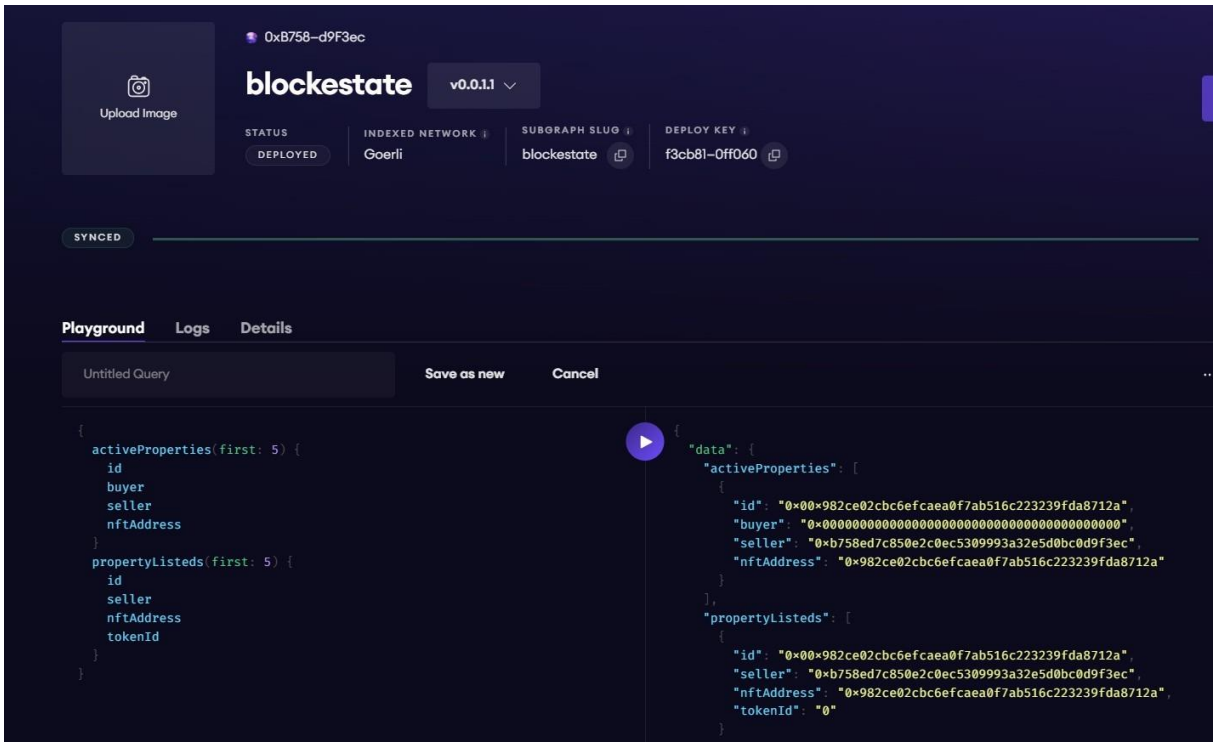

Figure 4.2.12 the graph playground and dashboard, a graphql query returns the needed results

4.3 Frontend with NextJS and Metamask

NextJS was the frontend framework that was used for this project. The easier routing system and the ability to render static web pages were the main reasons for choosing this framework instead of React. For styling, react-bootstrap and web3-ui-kit was heavily used. Metamask was successfully integrated to the frontend, and Moralis was used for easier handling of features and to call functions of the backend smart contracts.

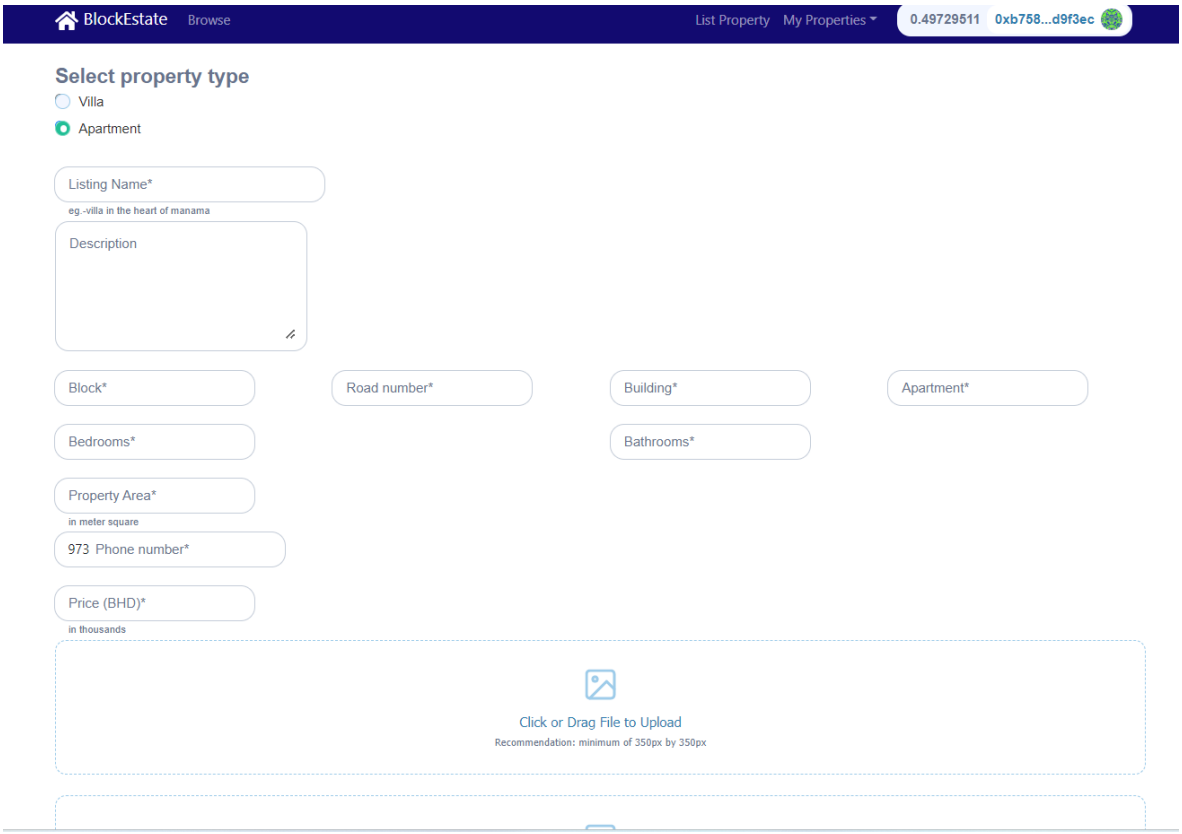

Figure 4.3.1 Form for the user the list a property on BlockEstate

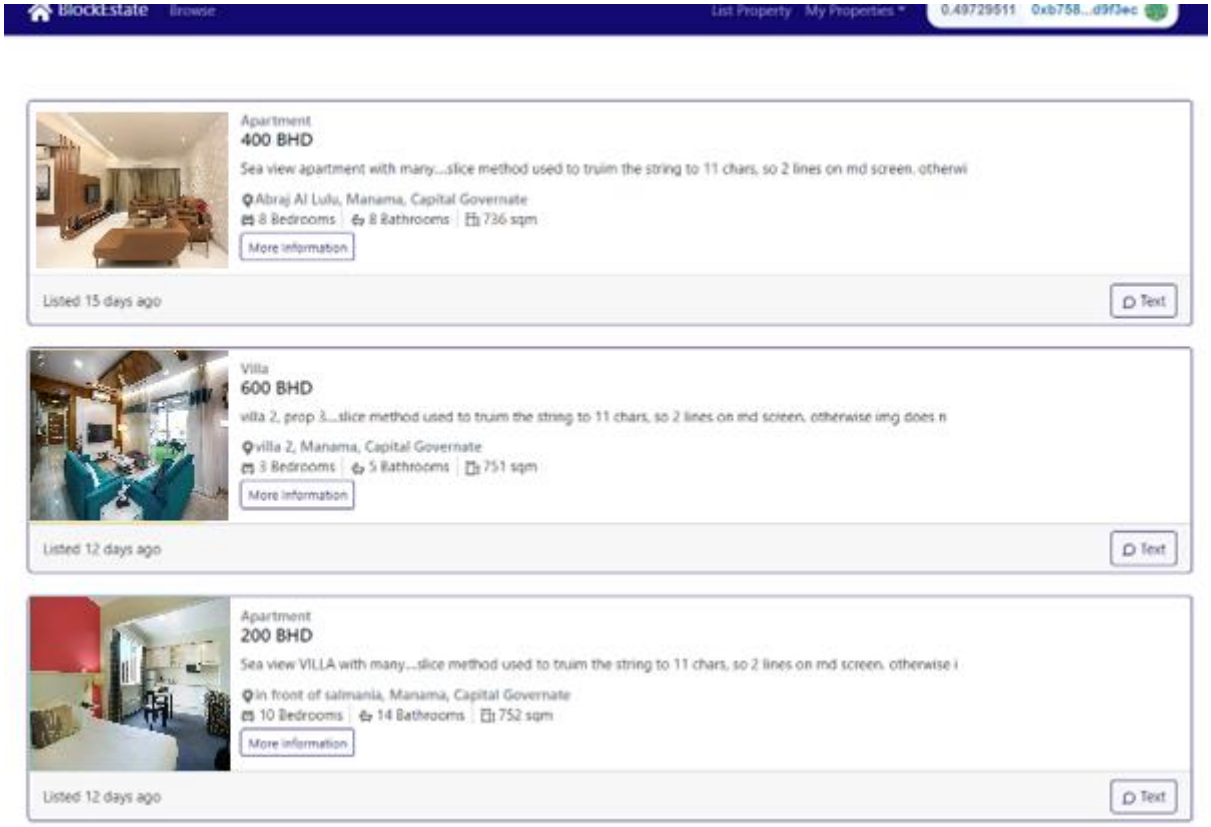

Figure 4.3.2 Browsing window of various properties

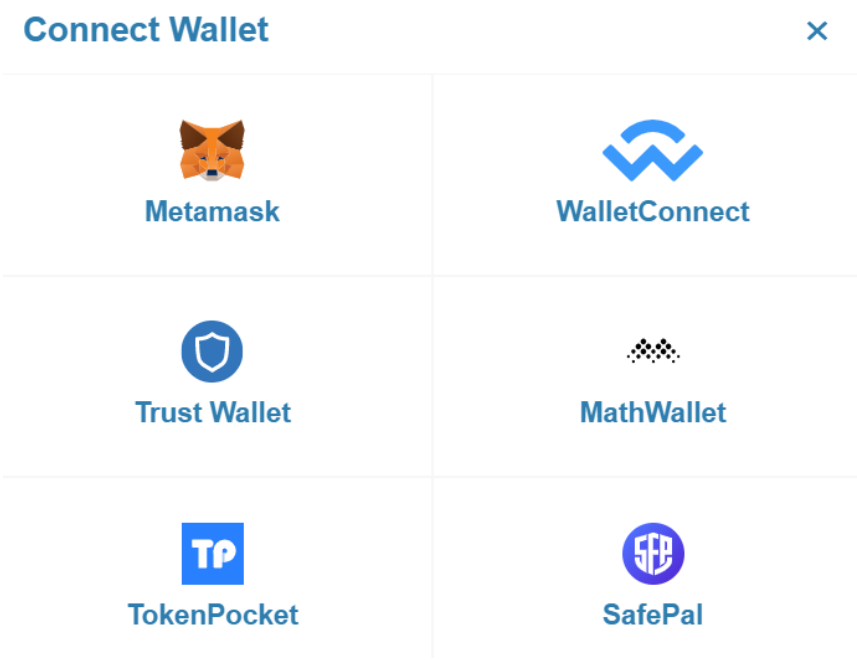

Figure 4.3.3 Multiple Connect Wallet Options

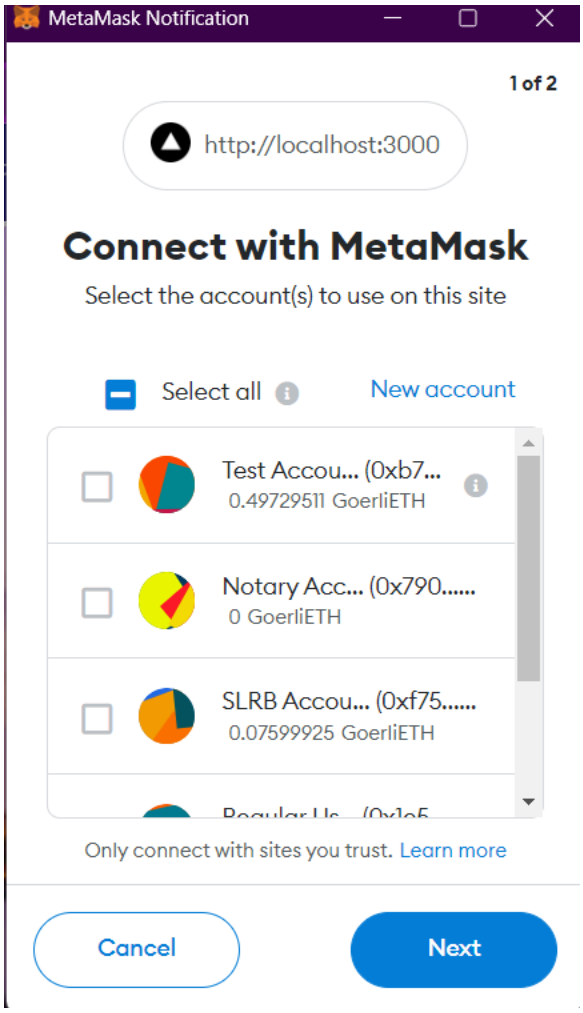

Figure 4.3.4 Using metamask to access various features like logging in, property listing and more

Chapter 5
Conclusion and Future Work

To conclude, learning about blockchain and decentralization were the main goals of this research. Since the future of the internet is most likely looking forward to decentralization, this research was just small baby steps to explore this world of web3 and blockchain. Real estate has always been one of the top markets of the modern economy, but the current situation involves many third parties, which reduces the profit, and is time-consuming, with tons of paperwork. Moreover, fraud and illegitimate transactions are quite common as well. Therefore, implementing real estate systems in the decentralized web3 and NFT workspace is a practical solution for the future. Replacing traditional paper certificates with NFT is one of the best use cases of the NFTs itself, which is commonly used now, just for digital art or games. Another aim of this project was to test the limits of decentralization, how far it can be done, and what limitations will it create. Scalability and speed are currently the major drawbacks of a completely decentralized system. Moreover, the lack of access to networks like IPFS in many web browsers is also a demerit. On the bright side, the development community is understanding the value of decentralization, and therefore, centralized companies and new startups will be encouraged to enable it in the foreseeable future.

Currently, this project is deployed in the Ethereum blockchain, as it's the number one network in blockchain space and has a huge development community backing it, with many tutorials and documentation related to EVM-based chains, particularly solidity. But unfortunately, ethereum is expensive, and the blockchain has high gas fees. So, one of the first changes will be switching to Polygon, which is also an EVM-based chain. The gas fees fees decrease drastically in Polygon mainnet, when compared to Ethereum. Another future work will be improving the frontend, as UI/UX experience is a great necessity to attract more users to the platform. The final and most challenging one perhaps, will be the implementation of fractional NFTs, which will enable multiple users to own a single property, by distributing the asset fractionally among the investors / spenders. This is really challenging as NFTs are in general a single token, with non-fungible literally in the name, but possible with some tweaking in the NFT workflow.

References

1.1 References

BBC , 2022. *What are NFTs and why are some worth millions?*. [Online] Available at: https://www.bbc.com/news/technology-56371912

Bendavid, N. & Harris, M., 2001. *Chicago Tribune*. [Online] Available at: https://www.chicagotribune.com/sns-mcdonalds-story.html

CB Insights Research, 2020. *Blockchain in Real Estate: How This Disrupts the| CB Insights.* [Online]
Available at: https://www.cbinsights.com/research/blockchainreal-estatedisruption/#:~:text=Tokenizing%20real%20estate%20assets%20refers,limit%20the%20risk%20of%20fraud

Chainlink, 2021. *Hybrid Smart Contracts Explained.* [Online] Available at: https://blog.chain.link/hybrid-smart-contracts-explained/

Chainlink, 2022. *Smart Contracts Introduction.* [Online] Available at: https://chain.link/education/smart-contracts

Eissler, E., 2022. *The Graph: Gathering Data from Other Blockchains to Facilitate Data Retrieval.* [Online]
Available at: https://atomicwallet.io/academy/graph-grt-guide [Accessed 18 11 2022].

Faiaz, Z., 2021. *Attacks, land grabs leave Bangladesh's Indigenous groups on edge.* [Online] Available at: https://landportal.org/node/100296

Grassegger, H. & Krogerus, M., 2017. *The Data That Turned the World Upside Down.* [Online]
Available at: https://www.vice.com/en/article/mg9vvn/how-our-likes-helped-trump-win

IPFS: Interplanetary file storage!. 2018. [Film] s.l.: Simply Explained.

IPFS, 2022. *Mint an NFT with IPFS.* [Online]
Available at: https://docs.ipfs.tech/how-to/mint-nfts-with-ipfs/ [Accessed 4 12 2022].

Jyotsna, Y. & Gampala, K., 2020. Blockchain for Real Estate. *researchgate,* Issue December 2020.

Keats, R., 2015. *US Banks Played a Pivotal Role in the 2008 Financial Crisis.* [Online] Available at: https://marketrealist.com/2015/11/us-banks-played-pivotal-role-2008-financial-crisis/

Nakamoto, S., 2008. Bitcoin: A Peer-to-Peer Electronic Cash System. *doi: 10.1007/,* Issue s10838-008-9062–0.

Pogue, D. & Miller, N., 2018. Sustainable real estate and corporate responsibility. In: *Routledge Handbook of Sustainable Real Estate.* s.l.:Routledge, pp. 19-36.

Potsides, A. et al., 2022. *IPFS Powers the Distributed Web.* [Online] Available at: https://ipfs.tech/ [Accessed 2 12 2022].

Siddiqui, Z. & Ghoshal, D., 2021. *Twitter blocks dozens of accounts on India's demand amid farm protests -sources.* [Online] Available at: https://www.reuters.com/world/twitter-blocks-dozens-accounts-indias-demand-amid-farm-protests-sources-2021-02-01/

What Is a Blockchain Oracle?. 2021. [Film] s.l.: Chainlink.

Zhou, J. & al, e., 2020. Applied Cryptography and Network. *Cham: Springer International Publishing AG.*